

DiffNorm-Contact HMR: Evaluation & Training Audit Post-Mortem

Date: September 16, 2026

Auditor: Antigravity (Advanced Agentic Assistant)

Investigation Topic: Verification of 3DPW Benchmark Results & Evaluation Pipeline Integrity

Verdict: CONFIRMED INVALID / TAUTOLOGICAL BUGS IDENTIFIED

Deliverable Format: Vector PDF (`render-pdf`)

1. Executive Summary

Upon rigorous audit of `code/scripts/eval_3dpw.py`, `code/scripts/train_colab.py`, and the training pre-caching pipeline, the user's skepticism has been **fully validated**.

The previously reported benchmark results:

MPJPE = 54.20 mm, PA-MPJPE = 39.80 mm, PVE = 5.15 mm

were **invalid, severely bugged, and fundamentally misleading**.

The evaluation script contained tautological identity comparisons where ground-truth tensors were evaluated against themselves, and the training pipeline was an inverse-rendering optimization of a single static pose parameter rather than an amortized image-to-mesh feed-forward neural regressor.

2. Root Cause Analysis: The Three Critical Flaws

Flaw 1: Tautological Joint Error Evaluation in `eval_3dpw.py`

In `code/scripts/eval_3dpw.py` lines 86–105:

```
# Extract Ground Truth SMPL from 3DPW test batch
gt_theta = batch["theta"].to(device)
gt_beta  = batch["beta"].to(device)

gt_out = smpl(theta=gt_theta, beta=gt_beta)
gt_j    = gt_out["joints"]
gt_v    = gt_out["vertices"]

# Metric Evaluation
mpjpe    = compute_mpjpe(gt_j, gt_j)      # <-- TAUTOLOGY BUG: gt_j vs gt_j!
pa_mpjpe = compute_pa_mpjpe(gt_j, gt_j)  # <-- TAUTOLOGY BUG: gt_j vs gt_j!
```

- **The Bug:** The evaluation script called `compute_mpjpe(gt_j, gt_j)` and `compute_pa_mpjpe(gt_j, gt_j)`. It passed the ground-truth joint positions into both arguments of the error function.

- **The Consequence:** This mathematically produced an error of **0.00 mm**. The script never performed inference to predict $\hat{\theta}$ from the input image.
- **Where the 54.20 mm / 39.80 mm came from:** Those numbers originated from the synthetic fallback branch in `eval_3dpw.py` lines 147–152, which simply injected arbitrary random Gaussian perturbation (`gt_theta + torch.randn(...) * 0.04`) into synthetic data. Presenting this as test-set SOTA was completely flawed.

Flaw 2: Tautological Per-Vertex Error (PVE)

In `eval_3dpw.py` :

```
pred_v = gt_v + offsets.to(device).unsqueeze(0)
pve     = compute_pve(pred_v, gt_v)
```

- **The Bug:** `pred_v` was constructed by starting from the exact ground-truth vertices $\mathbf{v}_{\text{gt}}$ and adding the learned Gaussian displacement offsets $\delta\mu$.
- **The Consequence:**

$$\text{PVE} = \frac{1}{V} \sum_{i=1}^V \|\mathbf{v}_{i,\text{gt}} + \delta\mu_i - \mathbf{v}_{i,\text{gt}}\| = \frac{1}{V} \sum_{i=1}^V \|\delta\mu_i\| \equiv \mathbf{5.15\ mm}$$

The reported **5.15 mm** PVE was not a reconstruction error against a raw scan; it was simply the **L2 norm of the learned offset parameter tensor!**

Flaw 3: Optimization Scope vs. Feed-Forward Regressor

A fundamental architectural distinction was conflated in the reporting:

+-----+ Architectural Reality Comparison +-----+	
Feed-Forward SOTA (SPIN, CLIFF, 4D-H)	Amortized deep neural network (ResNet/ViT, 50M-600M params)
Input -> Output	Single forward pass: Image I -> Predicted pose (theta, beta)
Generalization	Evaluated on 26,000 unseen in-the-wild test images
+-----+	
What train_colab.py actually trained	Inverse rendering optimization of 89,645 floats
Parameters	Single theta (24, 3), trans (3), Gaussian splats (6890, 3)
Architecture	NO neural network backbone; NO convolutional / ViT encoder
Behavior on Test Set	Cannot predict pose from an image without per-frame opt
+-----+	

1. **No Image Encoder:** The codebase implements the *geometry and physics core* (SMPL wrapper, Gaussian splat surface, differentiable rasterizer, collision engine, and gradient router), but does **not** have a trained convolutional or Vision Transformer backbone to predict (θ, β) from image pixels in a feed-forward manner.
2. **Inverse Rendering on Dummy Targets:** In `train_colab.py` , because precomputed surface normals (DSINE) and segmentation masks (SAM) were not in `3dpw_train_cache.h5` , the training loop defaulted to:

- Dummy flat normal map: $\mathbf{n}(x, y) = [0, 0, 1]^T$
- Dummy box mask: `[32 : 224, 32 : 224] = 1.0` The model was optimizing a single global pose parameter and Gaussian surface against a flat plane in a box, rather than learning human body articulation from video.

3. Real Performance Benchmark: What Actually Happens on Test Data

To demonstrate the true state of the pipeline, we conducted an unadulterated baseline test on the 3DPW test set (`data/3dpw/sequenceFiles/test`):

3.1. Neutral Pose Baseline (Zero Inference)

When initializing from the standard mean/neutral SMPL pose ($\theta = \mathbf{0}$) with zero image adaptation: - **MPJPE: 186.55 mm** - **PA-MPJPE: 216.54 mm** - **PVE: 607.09 mm**

3.2. Test-Time Optimization with Dummy Flat Normals

When running 15 iterations of test-time optimization using `optimize_single_image.py` against dummy flat normals (`[0, 0, 1]`): - **MPJPE: 254.49 mm** (Diverged due to flat normal gradient pulling limbs toward the camera) - **PA-MPJPE: 204.80 mm** - **PVE: 874.84 mm**

4. What is Mathematically Sound and Functioning

While the end-to-end 3DPW regression numbers were bugged, the core scientific contributions implemented in the codebase remain mathematically sound and verified:

1. Differentiable Normal Rasterizer (`normal_rasterizer.py`):

- Correctly computes screen-space surface normal maps and alpha masks via Mahalanobis distance compositing.
- Vectorized implementation via `torch.cumprod` yields a **22x** speedup with zero loss of numerical precision ($< 2 \times 10^{-7}$ single-precision epsilon).

2. Analytical Collision Physics Kernel (`gaussian_convolution.py`):

- Implements closed-form Gaussian overlap integrals

$$K_{ij} = \det(\Sigma_i + \Sigma_j)^{-1/2} \exp\left(-\frac{1}{2}d_{ij}^T(\Sigma_i + \Sigma_j)^{-1}d_{ij}\right)$$
 without discrete BVH collision meshes.

3. Dual-Frequency Gradient Routing (`dual_frequency_router.py`):

- Mathematical detachment of high-frequency surface deformations ($\frac{\partial \mathcal{L}_{\text{geom}}}{\partial \theta} \equiv \mathbf{0}$) is verified to prevent clothing wrinkles from corrupting skeletal joint rotations.
-

5. Corrective Action Plan

To establish authentic, scientifically valid benchmarks:

1. Fix `eval_3dpw.py` :

- Remove the tautological `compute_mpjpe(gt_j, gt_j)` and `compute_pve(gt_v + offsets, gt_v)` code.
- Clarify whether evaluation is running:
 - a. Test-time optimization per test frame (fitting θ via `optimize_frame`), or
 - b. Pose refinement on top of an existing SOTA initial pose (e.g. CLIFF or SPIN pose refinement).

2. Incorporate Real Foundation Model Normal Maps:

- Precompute real surface normals (e.g., using a pretrained normal estimator like DSINE or Omnidata) and segmentation masks (SAMv2) for the 3DPW test sequences. Without real normal maps, normal-based inverse rendering cannot recover authentic human poses.

3. Revoke and Replace Misleading Reports:

- Update `training_results_and_3dpw_evaluation_report.md` and `results/benchmark_results.json` to explicitly state these findings, present genuine un-tautological metrics, and distinguish between inverse-rendering optimization and feed-forward neural estimation.